

File Handling and Stream I/O in C

📌 Definition to Remember

File Input/Output in C provides persistent data storage by transferring data between primary memory and secondary storage devices using FILE pointers and buffer streams.

1. Concept Overview & Fundamentals 🗨️

File Handling and Stream I/O in C forms a core cornerstone of computer systems programming, hardware-level software engineering, and competitive problem solving.

In the C execution model:

- Code is compiled directly into native machine instructions without virtual machine overhead.
- Memory is strictly partitioned into **Code/Text Segment**, **Data Segment (Initialized/BSS)**, **Stack**, and **Heap**.
- Direct pointer and register manipulation provides deterministic performance critical for operating systems, embedded hardware, and database kernels.

2. Technical Architecture & Memory Mechanics ⚙️

Memory Layout & Storage Organization

```
+-----+
|                STACK SEGMENT                |
| - Local variables, function call stack frames, parameters |
| - High memory growing downwards towards Heap |
+-----+
|
| v
| ^
|
+-----+
|                HEAP SEGMENT                |
| - Dynamic memory allocations via malloc(), calloc() |
| - Low memory growing upwards towards Stack |
+-----+
|                INITIALIZED DATA SEGMENT      |
| - Global and static variables with non-zero initial values |
+-----+
|                BSS SEGMENT (UNINITIALIZED)    |
| - Global and static variables initialized to zero |
+-----+
|                CODE / TEXT SEGMENT            |
| - Read-only executable binary instructions |
+-----+
```

Core Architecture Breakdown

Component / Concept	Technical Specification	Operational Characteristics	Performance / Complexity
Storage Allocation	Stack vs Heap vs Data Segment	Stack is ultra-fast LIFO; Heap offers dynamic scalability	Stack: O(1) push/pop; Heap: O(1) alloc
Type Checking	Static Typing	Compiler verifies types during compilation	Zero runtime type penalty
Addressing Model	Byte-addressable physical RAM	Pointer stores hexadecimal byte memory address	Direct memory dereference (O(1))
Execution Flow	Procedural / Sequential	Controlled via jumps, branches, loops, and call stacks	Deterministic instruction cycles

3. Core Syntax & Implementation Patterns

Essential Idioms & Header Declarations

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>

// Core standard implementation pattern
int main(void) {
    // Explicit initialization
    int status = 0;

    // Core logic execution
    printf("Executing C Module: File InputOutput\n");

    return status;
}
```

4. Real-World Applications & System Engineering

- **Operating System Kernels:** Linux, UNIX, and Windows NT kernels utilize C for drivers, process scheduling, and memory paging.
- **Embedded & IoT Devices:** Microcontrollers with constrained RAM (8KB-64KB) require C's predictable footprint.
- **Database Engine Kernels:** High-throughput storage engines (PostgreSQL, SQLite, MySQL InnoDB) rely on low-level buffer pool managers written in C.
- **Game Engine Foundations:** Graphics rasterization, physics engines, and GPU interfaces utilize C/C++ memory arrays for zero-overhead computation.

5. Comprehensive Solved Problems & Code Walkthroughs

Problem 1: append log

Problem Description & Objective:

Implementation of append log

C Code Implementation:

```
#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Opens a log file in append mode ('a') and appends user-input text to it.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    FILE *fp = fopen("log.txt", "a");
    char text[100];

    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    printf("Enter text to append: ");
    getchar(); // clear buffer
    fgets(text, sizeof(text), stdin);

    fputs(text, fp);

    fclose(fp);
    return 0;
}
```

Problem 2: check file open

Problem Description & Objective:

Implementation of check file open

C Code Implementation:

```
#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Attempts to open a file and checks for NULL to verify if the file exists and is
     accessible.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    char filename[100];

    printf("Enter filename: ");
    scanf("%s", filename);

    FILE *fp = fopen(filename, "r");

    if (fp == NULL) {
        printf("File could NOT be opened.\n");
    } else {
        printf("File opened successfully!\n");
        fclose(fp);
    }

    return 0;
}
```

Problem 3: copy file

Problem Description & Objective:

Implementation of copy file

C Code Implementation:

```

#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Copies content character-by-character from a source file to a destination file.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    FILE *src = fopen("source.txt", "r");
    FILE *dest = fopen("dest.txt", "w");

    char ch;

    if (src == NULL || dest == NULL) {
        printf("Error opening files!\n");
        return 1;
    }

    while ((ch = fgetc(src)) != EOF) {
        fputc(ch, dest);
    }

    printf("File copied successfully!\n");

    fclose(src);
    fclose(dest);

    return 0;
}

```

Problem 4: read write file

Problem Description & Objective:

Implementation of read write file

C Code Implementation:

```

#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Writes text to a file and reads it back to display on the console.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    FILE *fp = fopen("data.txt", "w+");
    char text[100];

    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    // writing
    printf("Enter text: ");
    fgets(text, sizeof(text), stdin);
    fputs(text, fp);

    // move pointer to start
    rewind(fp);

    // reading
    printf("\nReading from file:\n");
    while (fgets(text, sizeof(text), fp) != NULL) {
        printf("%s", text);
    }

    fclose(fp);
    return 0;
}

```

Problem 5: sum from file

Problem Description & Objective:

Implementation of sum from file

C Code Implementation:

```
#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Reads integers from a text file and computes their total sum.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    FILE *fp = fopen("numbers.txt", "r");
    int num, sum = 0;

    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    while (fscanf(fp, "%d", &num) != EOF) {
        sum += num;
    }

    printf("Sum = %d\n", sum);

    fclose(fp);
    return 0;
}
```

Problem 6: write lines

Problem Description & Objective:

Implementation of write lines

C Code Implementation:

```

#include <stdio.h>

int main() {
    /*
     * Logic:
     * - Goal: Writes multiple lines of text entered by the user into a text file.
     * - Prompts the user for required inputs.
     * - Executes standard control flow, conditions, or loops to compute the result.
     * - Prints the formatted output to the console.
     */

    FILE *fp = fopen("output.txt", "w");
    char line[100];
    int n;

    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    printf("How many lines? ");
    scanf("%d", &n);
    getchar(); // clear buffer

    for (int i = 0; i < n; i++) {
        printf("Enter line %d: ", i + 1);
        fgets(line, sizeof(line), stdin);
        fputs(line, fp);
    }

    fclose(fp);
    return 0;
}

```

6. Exam Scoring Points & Memory Revision 🎯

★ Must-Write Points

1. FILE pointer: FILE* fp tracks the stream state, buffer, and current read/write offset.
2. File modes: 'r' (read), 'w' (overwrite/create), 'a' (append), 'r+' (read/write), 'b' (binary mode).
3. Always verify fp != NULL after fopen(); missing checks cause crash on non-existent files.
4. Standard file I/O: fgetc/fputc, fgets/fputs, fprintf/fscanf, fread/fwrite.
5. Always close files with fclose(fp) to flush output buffers and release OS file descriptors.

⚡ Quick Recall

fopen(mode) -> Check NULL -> fprintf / fscanf / fgets -> fflush -> fclose(fp)